TASK: 10

BUFFER OVERFLOW
&
LONG PASSWORD DOS ATTACK

Surendra Gabbani

(BUFFER OVERFLOW)

What is a buffer overflow?

A buffer overflow happens when a program writes more data into a fixed-size memory buffer than it was allocated to hold. The extra bytes overwrite adjacent memory (stack frames, heap metadata, control data), which can cause crashes or — in worst cases — arbitrary code execution.

Common types

- Stack buffer overflow — writing past a local (stack) buffer. Can overwrite saved registers, return address, stack canary, etc.
- Heap buffer overflow — overwriting heap memory and allocator metadata. Can be used for arbitrary memory corruption.
- Global/static buffer overflow — overwrite global variables/data.
- Off-by-one — writing exactly one too many bytes; sometimes exploitable in subtle ways.

How They Occur :

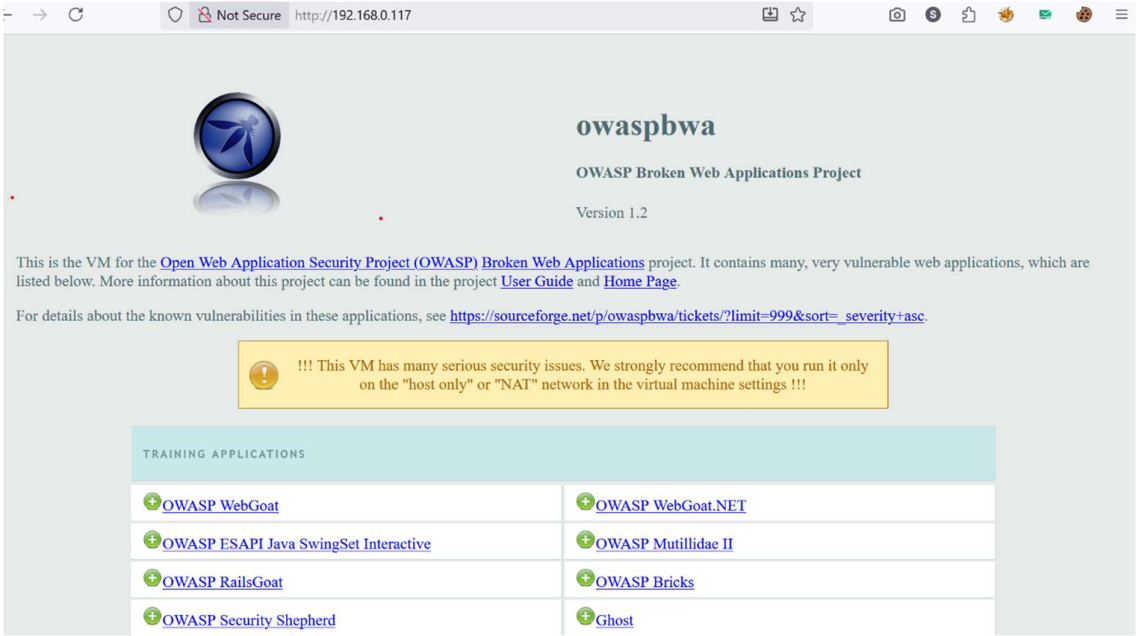
- Unsafe APIs and missing bounds checks (gets, strcpy, sprintf).
- Incorrect length calculations and assumptions about input.
- Parsing errors, malformed files, and user-supplied data. Important Headers

Detection & analysis tools:

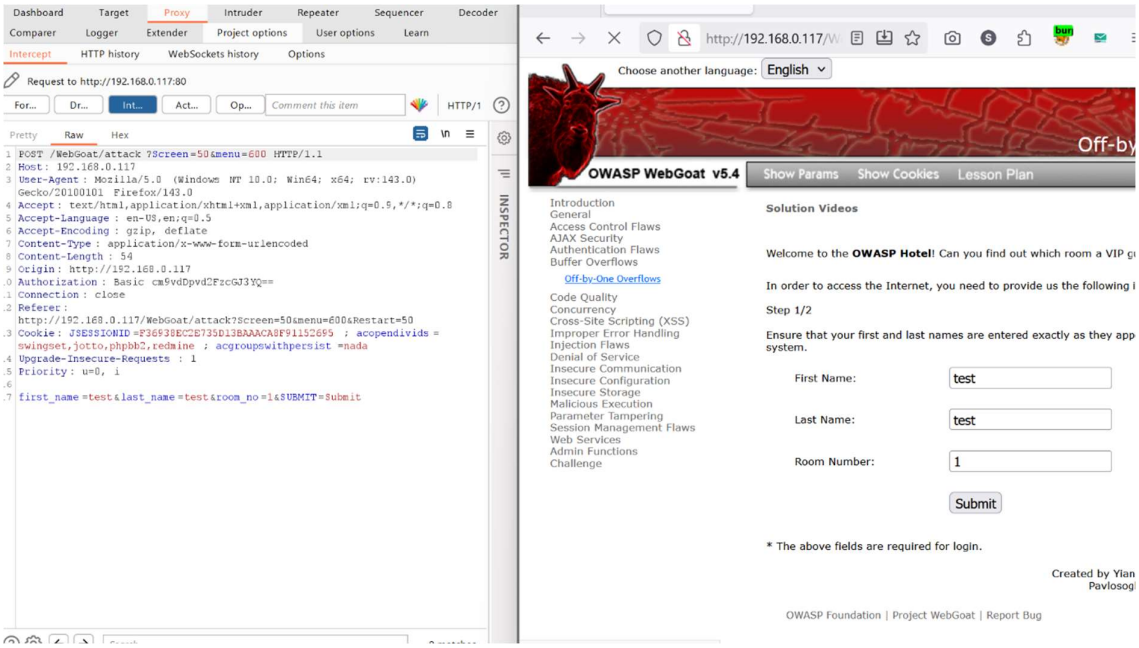
- **Static analysis**: clang/clang-static analyzer, cppcheck, Coverity.
- **Dynamic analysis / fuzzing**: AFL, libFuzzer, honggfuzz.
- **Runtime sanitizers**: ASan (AddressSanitizer), UBSan (UndefinedBehaviorSanitizer), MSan.
- **Memory debuggers**: Valgrind (memcheck).
- **Debuggers**: gdb (for inspection), but don't use it to craft exploits for wrongdoing.

LAB

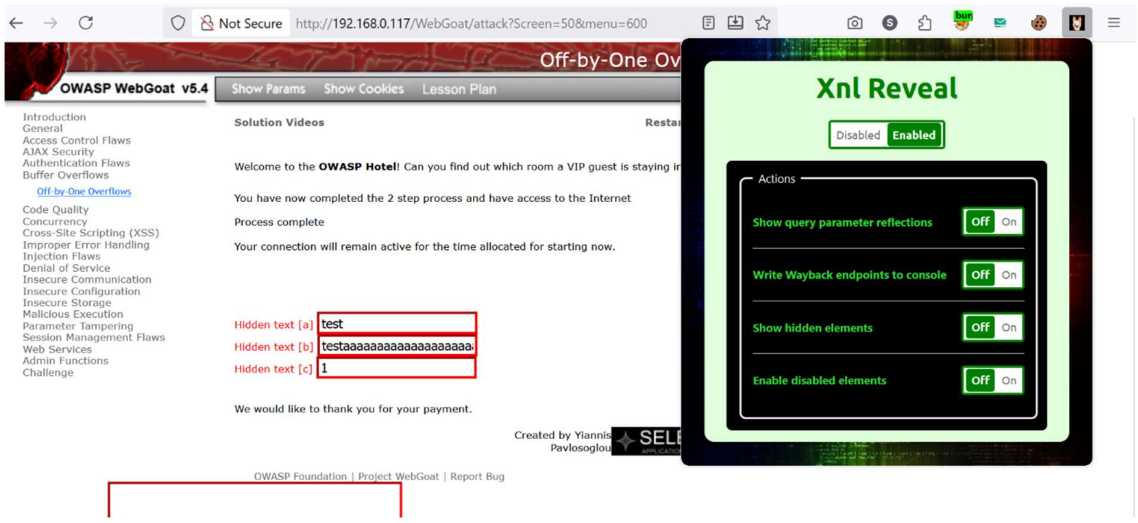
STEP 1: Open this OWSAP website and click on WebGoat



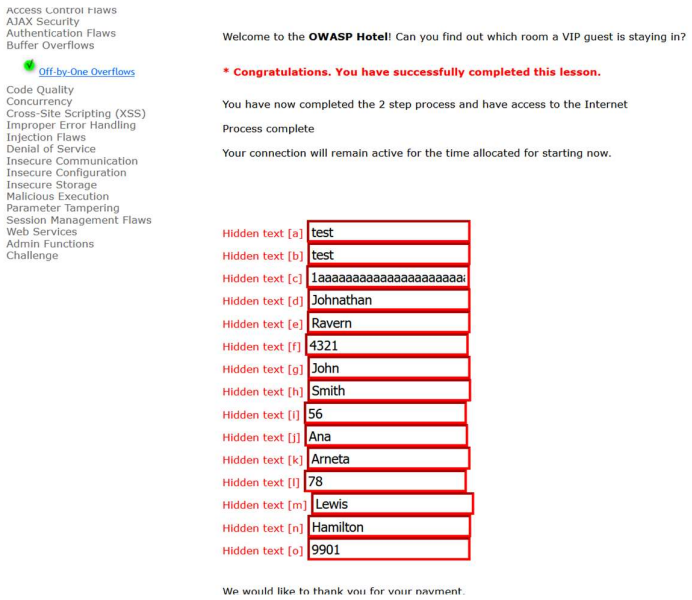
STEP 2: login into website and capture the request in (Burpsuite)



STEP 3: Add XNL Reveal Extension, setup long string data into it.



STEP 4: Finally data has spilled out and we can retrieve all credentials.



(LONG PASSWORD DOS ATTACK)

What is Long Password DOS Attack:

- A Long-Password DoS attack is where an attacker repeatedly submits extremely long password strings (or very large authentication payloads) to an authentication endpoint. The goal is to force the server to waste CPU, memory, or hashing resources (e.g., bcrypt/scrypt/PBKDF2) performing expensive work on each attempt, eventually degrading service or exhausting resources.Limitations:

Attack Works (step-by-step):

- Attacker crafts very large password field (kB → MB).
- Sends repeated login requests (from one or many IPs).
- Server accepts the request, allocates memory and performs parsing + hashing.
- Resources consumed; legitimate users degrade or fail.

Why it hurts:

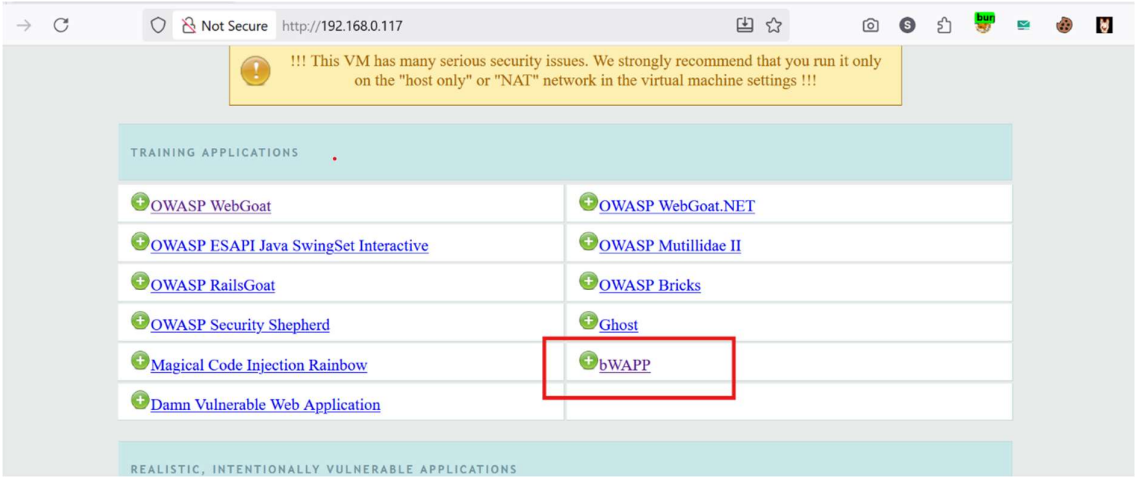
- Password hashing (bcrypt/scrypt/PBKDF2/Argon2) is intentionally slow — good for security, but expensive if abused.
- Very large input sizes can cause excessive CPU, large memory allocations, or trigger pathological behavior in parsing libraries.
- If the service does full hashing/processing for each unauthenticated request, attackers can amplify small request rates into big resource costs.

Defensive principles (high level):

- **Fail fast** — reject obviously invalid input *before* expensive work.
- **Limit sizes** — enforce maximum password length early (request parsing/WAF/load-balancer).
- **Rate limit and throttle** — per IP, per account, per endpoint.
- **Progressive mitigation** — CAPTCHAs, exponential backoff, lockouts for suspicious activity.
- **Do expensive work *only* after minimal checks pass.**

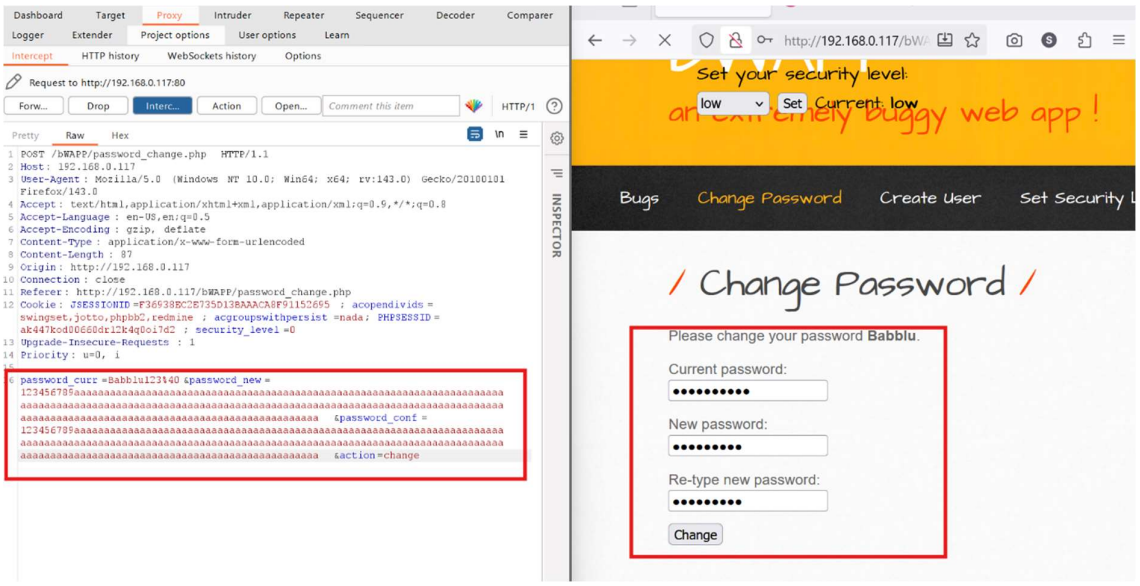
LAB

STEP 1: Open this OWSAP website and click on bWAPP

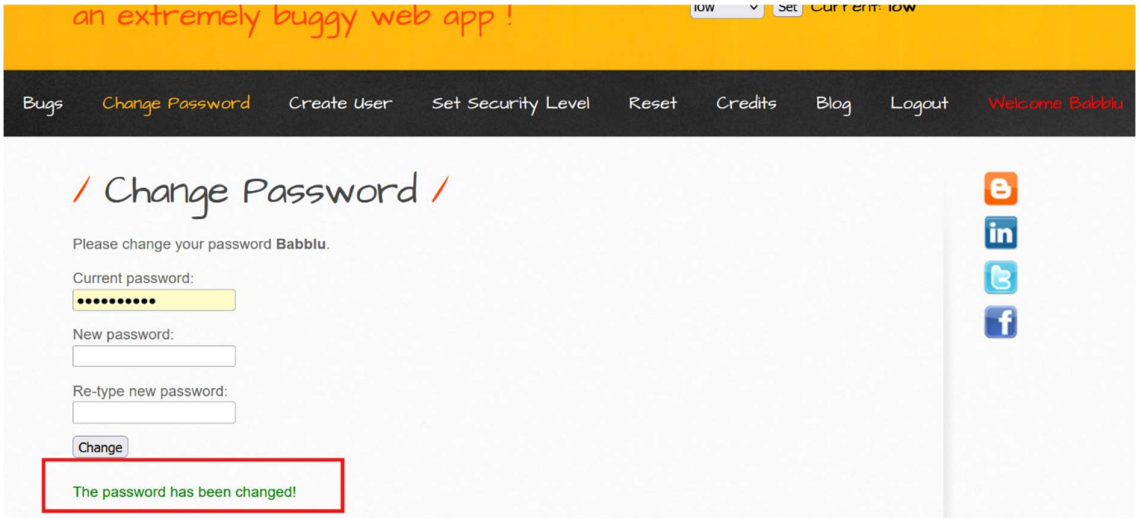


STEP 2:

- Login with credentials and change the password
- Capture the request in Burpsuite
- By sending a very long enough string to change the password


Dashboard
Target
Proxy
Intruder
Repeater
Sequencer
Decoder
Comparer
Logger
Extender
Project options
User options
Learn
Intercept
HTTP history
WebSockets history
Options
Request to http://192.168.0.117:80
Forw...
Drop
Interc...
Action
Open...
Comment this item
HTTP/1
Pretty
Raw
Hex
POST /bWAPP/password_change.php HTTP/1.1
Host: 192.168.0.117
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:143.0) Gecko/20100101 Firefox/143.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 87
Origin: http://192.168.0.117
Connection: close
Referer: http://192.168.0.117/bWAPP/password_change.php
Cookie: JSESSIONID=F36938EC2E735D13BAAACA8F91152695 ; acopendivids=swingset,jotto,phpbb2,redmine ; acgroupswithpersist=nada; PHPSESSID=ak447kodU06Gdr12k4q0o17d2 ; security_level=0
Upgrade-Insecure-Requests: 1
Priority: u=0, i
password_curr=Babblu123%40&password_new=123456789aaaa

STEP 3: (Password change successful)



References & resources:

- OWASP guidance (authentication and input limits)
- NGINX docs: client_max_body_size, header buffers
- Password hashing best practices (Argon2, bcrypt)

THANK YOU